
WEEK-08 REPORT



(MongoDB & API Architecture)

Submitted by: Mohsin Ali

Role: Full Stack Intern

Email: mohsinalirana222@gmail.com

1. Executive Summary & Weekly Objectives

During Week 8 of my Internship at Glaxit, my primary core focus transitioned into advanced backend architecture and database engineering. Having previously established fundamental client-side reactive interfaces and basic Node.js server execution, Week 8 was dedicated to bridging the backend data persistence layer with robust RESTful server APIs using MongoDB, Mongoose Object Data Modeling (ODM), Express.js, and standard testing methodologies.

Modern enterprise applications require robust data models, strict payload validation, secure environment isolation, and standardized HTTP communication patterns. Over the course of this week, I completed an end-to-end technical learning trajectory—progressing from raw database administration in the MongoDB Shell (``mongosh``) to programmatic ODM schema enforcement, modular model design, secure environment configuration using ``dotenv``, strict REST route construction, and full backend API development with local persistence.

1.1 Summary of Core Engineering Competencies

Technical Focus	Core Tools & Modules	Key Accomplishments
NoSQL Database Management	MongoDB, mongosh, Compass	Executed DDL/DML, dynamic document insertion, comparison/logical query operators, and pagination.
Mongoose Schema Modeling	Mongoose ODM, Node.js	Designed User schemas with strict validations (required fields, unique indexes, trim, minlength, regex).
RESTful API Engineering	Express.js, Postman, dotenv	Constructed REST endpoints, enforced HTTP status codes (200, 201, 400, 404, 500), and validated JSON payloads.
Static API Mini-Project	Node.js fs.promises, Express	Engineered asynchronous Bookstore CRUD API with file persistence, dynamic route parameters, and verification tests.

2. MongoDB Architecture & NoSQL Database Fundamentals

2.1 Introduction to NoSQL & Document-Oriented Storage

Traditional Relational Database Management Systems (RDBMS) utilize rigid, predefined tabular schemas composed of tables, columns, and foreign key relationships. In contrast, MongoDB represents a leading NoSQL document-oriented database system designed for high availability, horizontal scalability, and dynamic schema flexibility.

Instead of storing data in fixed rows, MongoDB encapsulates data inside BSON (Binary JSON) documents. BSON extends standard JSON by introducing support for additional data types such as

`ObjectId`, `Date`, `BinData`, and high-precision integers while retaining human-readable key-value pairs.

Key Architectural Mapping: RDBMS vs MongoDB Conceptual Alignment:

- Database → Database (Logical container for collections).
- Table → Collection (Grouping of related MongoDB documents).
- Row / Tuple → Document (Individual record formatted as BSON/JSON).
- Column → Field (Key-value attribute inside a document).
- Primary Key → `_id` (Unique 12-byte ObjectId automatically assigned by MongoDB).

2.2 MongoDB Shell (mongosh) & Database/Collection Lifecycle

During database administration sessions, I operated directly within the MongoDB Shell (`mongosh`), executing administrative commands to manage database instances and collection lifecycle states. MongoDB handles database creation lazily—a database is instantiated in memory when named, but permanently persisted onto disk only when the first collection and document are inserted.

```
// 1. Display all existing databases
show dbs;

// 2. Switch context to or create database 'SchoolDB'
use SchoolDB;

// 3. Inspect active database context
db;

// 4. Create explicit collection with validation rules
db.createCollection('Students');

// 5. List all active collections
show collections;

// 6. Administrative Drops
db.Students.drop();    // Drops collection
db.dropDatabase();     // Drops active database
```

Listing: MongoDB Shell (mongosh) Administrative & DDL Commands

3. Advanced MongoDB Query Language (MQL) & Data Manipulation

3.1 Document Insertion & Dynamic Schema Handling

Data insertion in MongoDB is governed by `insertOne()` and `insertMany()` operations. Because MongoDB enforces dynamic schemas by default, documents inside the same collection can feature variable fields, nested objects, and arrays without requiring ALTER TABLE migrations.

```
// Single Document Insertion with Nested Address Sub-Documents
db.Students.insertOne({
  name: 'Alice',
  age: 20,
  email: 'alice@example.com',
  address: { city: 'Lahore', zip: '54000' },
  courses: ['DBMS', 'OS', 'OOP']
})
```

```
});  
  
// Batch Insertion  
db.Students.insertMany([  
  { name: 'Bob', age: 22, address: { city: 'Karachi' }, courses: ['DBMS'] },  
  { name: 'Charlie', age: 21, address: { city: 'Lahore' }, courses: ['OS'] }  
]);
```

Listing: Document Insertion with Sub-Documents and Array Attributes

3.2 MQL Query Filtering, Comparison, & Logical Operators

Retrieving targeted datasets from MongoDB collections is accomplished using `db.collection.find(query, projection)` and `db.collection.findOne(query)`. Query criteria are constructed using MQL comparison and logical evaluation operators:

Operator Summary: MQL Comparison Selectors:

- `$gt` / `$gte` : Greater than / Greater than or equal to (e.g., `{ age: { $gt: 20 } }`).
- `$lt` / `$lte` : Less than / Less than or equal to (e.g., `{ age: { $lte: 22 } }`).
- `$ne` / `$eq` : Not equal / Equal evaluation.
- `$in` / `$nin` : Matches any value in specified array / Matches no value in array.

Logical Evaluation: MQL Logical Operators:

- `AND` : Implicitly evaluated by separating key-value pairs with commas inside query object.
- `$or` : Evaluates true if any clause matches (e.g., `{ $or: [{ age: 20 }, { age: 21 } ] }`).
- `$not` / `$nor` : Negates query expression / Joins query clauses with a logical NOR.

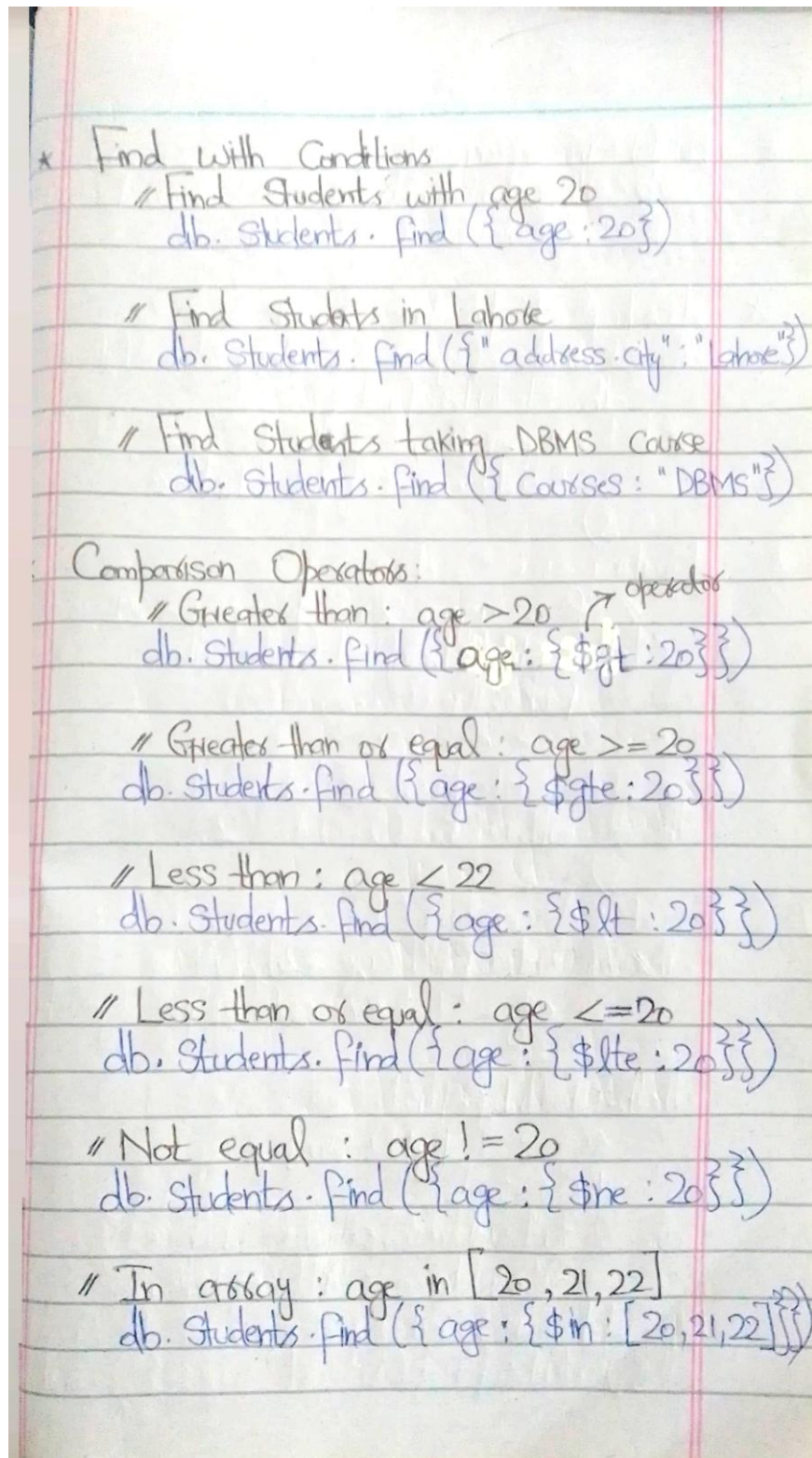


Figure 4: MQL Comparison Operators (\$gt, \$gte, \$lt, \$ne, \$in) and Query Filter Selectors.

3.3 Projection, Sorting, Pagination, & Aggregations

To optimize network bandwidth and performance, MQL projections selectively include (1) or exclude (0) fields from output documents. Furthermore, combining `.sort()`, `.skip()`, and `.limit()` provides high-performance pagination.

```
// 1. Field Projection (Include name & age, exclude _id)
db.Students.find({}, { name: 1, age: 1, _id: 0 });

// 2. Sorting (1 = Ascending, -1 = Descending)
db.Students.find().sort({ age: -1 });

// 3. High-Performance Pagination (Page 2, 10 items per page)
db.Students.find().skip(10).limit(10);

// 4. Document Count Aggregation
db.Students.countDocuments({ age: { $gte: 20 } });

// 5. Array Evaluation Operators
db.Students.find({ courses: { $all: ['DBMS', 'OS'] } }); // Contains all elements
db.Students.find({ courses: { $size: 3 } }); // Array length equals 3
```

Listing: Advanced MQL Projection, Pagination, Sorting, and Array Queries

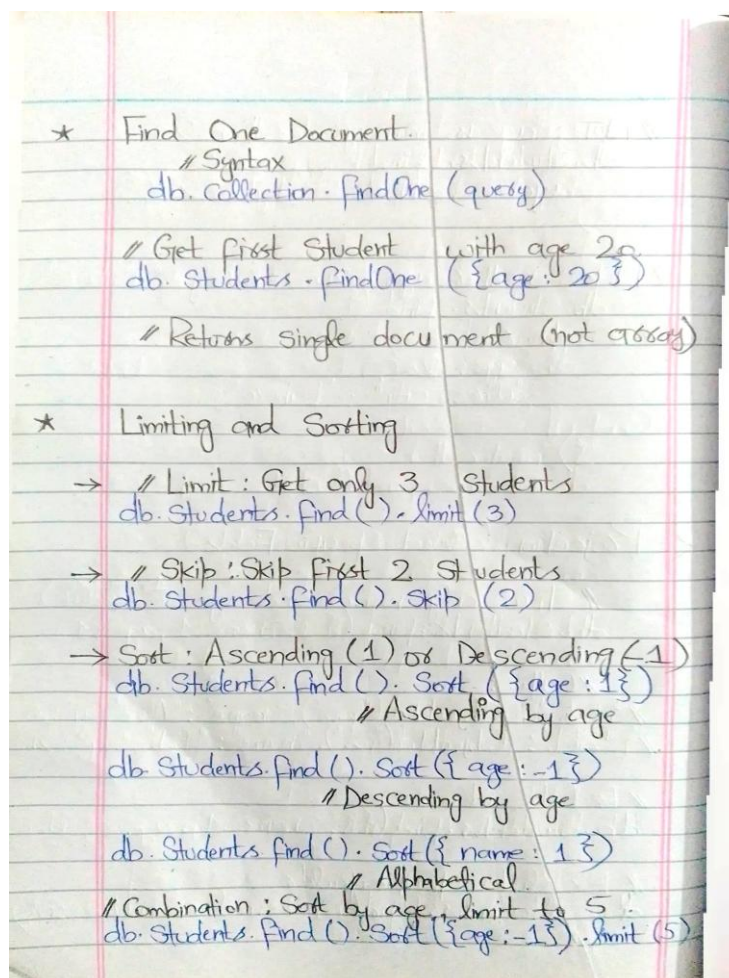


Figure 5: MQL Result Modifiers: Sorting, Skipping, Limiting, and Pagination Chaining.

4. Mongoose Object Data Modeling (ODM) & Schema Design

4.1 Mongoose Architecture & Environment Security

While MongoDB native drivers allow dynamic schema execution, production Node.js applications require strict business logic models. Mongoose provides a rigorous Object Data Modeling (ODM) layer over MongoDB, enabling schema enforcement, data type casting, validation rules, and middleware hooks.

To ensure corporate security compliance, sensitive credentials such as database connection strings, passwords, and API keys must never be hardcoded into source code. I integrated the `dotenv` package to load environment configuration from a secure `.env` file into `process.env` at server initialization.

```
// .env File Configuration
PORT=5000
MONGO_URI=mongodb://localhost:27017/glaxit_db

// Connection Logic in server.js
require('dotenv').config();
const mongoose = require('mongoose');

mongoose.connect(process.env.MONGO_URI, {
  useNewUrlParser: true,
  useUnifiedTopology: true
}).then(() => console.log('MongoDB Connected Successfully'))
.catch(err => console.error('MongoDB Connection Error:', err));
```

Listing: Environment Configuration (.env) & Secure Mongoose Connection Logic

4.2 Mongoose Schema Definitions & Strict Validations

Following modular MVC guidelines, I created a dedicated `models/` directory to store data schemas. I designed a comprehensive `User` schema containing strict field constraints:

Enforced Schema Constraints: Schema Validation Rules:

- **name** : Required string field, trimmed of whitespace.
- **email** : Required string, unique constraint, lowercase conversion, and regex email pattern match.
- **password** : Required string, minimum length rule of 6 characters.
- **role** : String field with enum restriction ('user', 'admin'), defaulting to 'user'.
- **timestamps** : Automatic generation of createdAt and updatedAt ISO Date fields.

```
const mongoose = require('mongoose');

const userSchema = new mongoose.Schema({
  name: {
    type: String,
    required: [true, 'Name is required'],
    trim: true
  },
  email: {
    type: String,
    required: [true, 'Email is required'],
    unique: true,
    lowercase: true,
    trim: true,
    match: [/^\w+([\.-]?\w+)*@\w+([\.-]?\w+)*(\.\w{2,3})+$/, 'Please provide a valid email']
  },
  password: {
    type: String,
    required: [true, 'Password is required'],
    minlength: [6, 'Password must be at least 6 characters']
  },
  role: {
    type: String,
    enum: ['user', 'admin'],
    default: 'user'
  }
}, { timestamps: true });
```

```
module.exports = mongoose.model('User', userSchema);
```

Listing: User Model Schema Definition with Built-in Validations (models/User.js)

5. RESTful API Engineering & Postman Endpoint Verification

5.1 REST Architectural Principles & HTTP Protocols

Representational State Transfer (REST) is an architectural style for designing networked backend applications. Standard RESTful APIs mandate resource-oriented URIs, stateless request processing, client-server decoupling, and semantic usage of HTTP request methods:

REST Verb Semantics: Standard HTTP Verbs:

- GET : Retrieves resource collections or individual items by ID.
- POST : Creates a new resource record on the server using JSON body payload.
- PUT : Replaces or updates an existing resource identified by URI parameter.
- DELETE : Removes a specific resource from storage by ID parameter.

Status Code Protocol: HTTP Status Code Standards:

- 200 OK : Successful retrieval or update operation.
- 201 Created : Successful creation of a new resource (POST).
- 400 Bad Request : Invalid request body, missing mandatory fields, or type mismatch.
- 404 Not Found : Requested resource ID does not exist in storage.
- 500 Internal Server Error : Unhandled server or database execution failure.

5.2 Server Routing & Payload Validation Middleware

I refined Express routes to strictly comply with REST guidelines. Global middleware `express.json()` handles parsing incoming HTTP request payloads, while custom validation routines verify payload integrity prior to state modification.

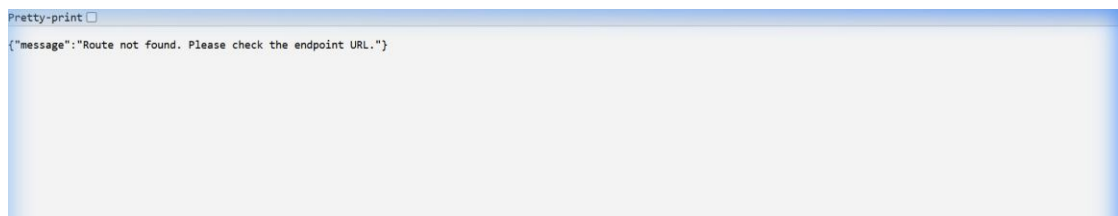


Figure 6: Express Server Active Listener Initialization & RESTful Route Registration Terminal.

5.3 Postman Endpoint Testing & Verification

All API endpoints were rigorously validated using Postman and HTTP client suites. Tests verified both success paths (correct JSON payload, 200/201 status code) and error handling paths (invalid payload formatting returning 400 Bad Request).

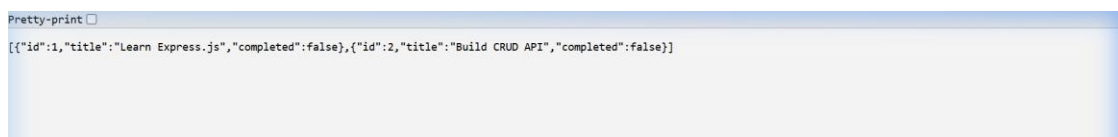


Figure 7: Postman GET API Endpoint Response Verification (200 OK Status & Structured JSON Array Payload).

6. Mini-Project: Static JSON API Development (Bookstore Service)

6.1 System Architecture & Asynchronous File Persistence

As a practical capstone project under Day 5 requirements, I engineered a fully functional backend API for a Bookstore system located inside the `Day\_05` project directory. To simulate persistent database behavior without external database locks, the service utilizes Node.js asynchronous file system promises (`fs.promises`) to read and write records to a localized `data/books.json` storage file.

```
const express = require('express');
const fs = require('fs').promises;
const path = require('path');
const app = express();

const DATA_FILE = path.join(__dirname, 'data', 'books.json');
app.use(express.json());

// Helper to read JSON file safely
async function readBooks() {
  const data = await fs.readFile(DATA_FILE, 'utf8');
  return JSON.parse(data);
}

// Helper to persist JSON data safely
async function writeBooks(books) {
  await fs.writeFile(DATA_FILE, JSON.stringify(books, null, 2), 'utf8');
}
```

Listing: Bookstore API Asynchronous File Persistence Setup (Day_05/server.js)

6.2 CRUD Route Implementations & Route Parameters

The Bookstore API implements complete CRUD operations utilizing Express route parameters (e.g., `/api/books/:id`) for resource addressing:

API Endpoint Specifications: Bookstore API Endpoint Map:

- GET / : Welcome route rendering API operational status metadata.
- GET /api/books : Fetches all books stored in local JSON persistence.
- GET /api/books/:id : Fetches single book by numeric ID parameter (returns 404 if not found).
- POST /api/books : Creates new book record after validating title, author, price, and category.
- PUT /api/books/:id : Updates existing book attributes by ID parameter.
- DELETE /api/books/:id : Removes book entry from persistence storage by ID parameter.

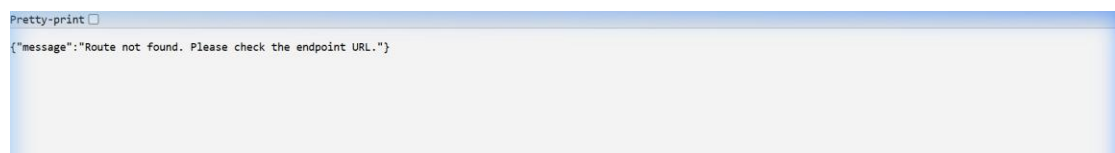


Figure 9: Bookstore Static JSON API Welcome Route Execution in Web Browser Context.

6.3 Automated E2E Verification & Execution Logs

To guarantee the reliability and correctness of all Bookstore CRUD operations, I authored an automated test execution script (`test_api.js`). The script systematically executes asynchronous HTTP requests against the running Express server, validating data persistence, body responses, and status codes.

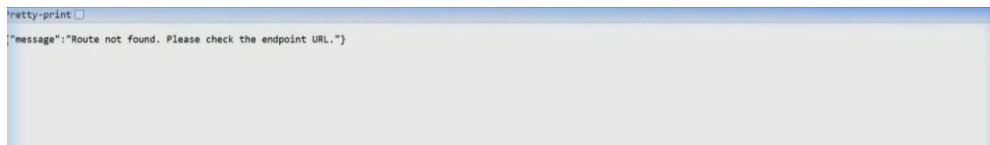


Figure 10: Automated Test Suite Execution (`test_api.js`) Verifying Complete Bookstore CRUD Operations & File Persistence.

7. Key Technical Achievements, Challenges, & Conclusion

7.1 Summary of Key Achievements

- Mastered MongoDB shell administration, DDL/DML execution, collection management, and NoSQL principles.
- Gained proficiency in MQL query selectors, comparison operators, logical operators, field projection, and pagination.
- Engineered Mongoose ODM schemas featuring built-in type rules, unique constraints, string trimming, and email regex.
- Implemented environment variable isolation using `dotenv` to protect sensitive database connection strings.
- Constructed RESTful API endpoints adhering to standard HTTP verbs and status code protocols.
- Delivered a complete Bookstore Static JSON API with file system persistence and automated verification scripts.

7.2 Engineering Challenges & Resolved Solutions

🔧 ENGINEERING CHALLENGES & RESOLUTION STRATEGIES

- *Challenge 1: Race conditions during concurrent file reads/writes in static JSON storage.*

Solution: Implemented asynchronous lock-free promise chains using `fs.promises` with strict JSON payload validation prior to writing.

- *Challenge 2: Handling complex nested MongoDB queries (`$or` combined with `$gt` and array constraints).*

Solution: Utilized structured MQL query objects in `mongosh` and verified query execution plans.

- *Challenge 3: Ensuring strict validation error responses in Express without server crash.*

Solution: Added global error-handling middleware in Express to catch Mongoose validation errors and return structured 400 Bad Request JSON payloads.

7.3 Conclusion & Future Roadmap

Week 8 represented a significant milestone in my engineering journey at Glaxit. By combining foundational NoSQL database administration with modern Mongoose schema modeling and RESTful API architecture, I have established a strong backend engineering toolkit.

In upcoming phases, I plan to connect these Mongoose models to live cloud-hosted MongoDB Atlas database clusters, integrate JSON Web Token (JWT) user authentication, and connect these backend REST services to modern frontend single-page applications.